Rapport de TP — Séance 2 : Service d'Ingestion IoT Agricole

Cours : Architecture & Ingestion Big Data |
Sujet : Construction d'un modèle objet résilient et d'un validateur cumulatif polymorphique

1. Contexte & Objectifs du TP

Dans le cadre de la modernisation d'une exploitation agricole, des capteurs IoT ont été déployés afin de surveiller la météo et de piloter automatiquement l'irrigation des cultures. Ces capteurs transmettent des données environnementales critiques (température, humidité de l'air et du sol) ainsi que l'état opérationnel des infrastructures (état de la pompe, débit d'irrigation).

L'objectif de ce TP est de concevoir la couche logicielle d'ingestion locale capable de :

- **Garantir l'intégrité des données** à l'aide de contrats d'interface stricts (Dataclasses Python).
- **Détecter et isoler les anomalies** via un moteur de validation cumulative et polymorphique.
- **Sécuriser la traçabilité** en implémentant des mécanismes de désinfection (sanitization) et de masquage des secrets au niveau des logs (conformité RGPD / Sécurité).
- **Valider la résilience du système** par des tests d'intégration automatisés (asserts).

2. Architecture des Fichiers du Projet

Le projet respecte scrupuleusement l'arborescence modulaire demandée, ce qui sépare de manière étanche les responsabilités :

- **models.py :** Définition des structures de données et de la sérialisation.
- **validators.py :** Logique des validateurs polymorphiques et cumulatifs.
- **main.py :** Point d'entrée, configuration des logs et scénarios de test.

3. Implémentation Technique

3.1. Les Contrats de Données (models.py)

Nous avons utilisé des dataclasses Python fortement typées. Ce choix technique par rapport à des dictionnaires classiques permet de valider la structure des objets dès la compilation/interprétation et d'éviter les erreurs d'accès aux clés manquantes.

- **ValidationError :** Gelée (frozen=True) pour garantir son immuabilité lors du transport des anomalies.

- **SensorReading :** Représente une mesure unitaire. Elle intègre une méthode d'auto-validation __post_init__ pour interdire un identifiant de capteur (sensor_id) vide.
- **IngestRequest & IngestResponse :** Encapsulent les échanges réseaux globaux. Par sécurité, la méthode to_dict() de IngestRequest exclut explicitement la clé API afin d'éviter toute fuite accidentelle dans les systèmes de stockage secondaires.

3.2. Moteur de Validation Polymorphique (validators.py)

Afin de rendre le système pleinement extensible (ajout futur de nouvelles règles sans modifier le code existant), nous avons implémenté le Design Pattern Strategy. Une classe de base abstraite Validator force l'implémentation d'une méthode validate(). Trois types de validateurs concrets ont été développés :

- **1. Structurel (RequiredFieldsValidator) :** S'assure que les champs obligatoires sont présents et non vides.
- **2. De surface (RangeValidator) :** Valide que les métriques physiques respectent des limites plausibles (ex: humidité entre 0 et 100%).
- **3. Métier / Cohérence (ConsistencyValidator) :** Vérifie l'état logique du système (si la pompe est active, le débit doit être strictement positif).

La fonction run_validators accumule toutes les erreurs d'une même ligne en une seule passe, évitant l'effet tunnel où une erreur première en cache une autre sous-jacente.

3.3. Sécurité du Logging & Traçabilité (main.py)

Le logging a été configuré pour répondre précisément aux exigences industrielles de sécurité informatique :

- **Masquage des Secrets :** La fonction mask_api_key applique un filtre de chaîne pour ne laisser visibles que les 4 derniers caractères d'une clé API (ex: ****1234).
- **Désinfection de l'entrée :** La fonction sanitize_for_log neutralise les injections de caractères invisibles (\n, \r, \t) qui pourraient falsifier l'affichage des logs (Log Injection attack) et tronque les données à 200 caractères pour prévenir le déni de service par saturation de disque.

4. Analyse du Jeu de Données de Test & Résultats

Le jeu de données simulé comportait 5 enregistrements conçus pour pousser le système dans ses retranchements :

Échantillon	Capteur	Statut attendu	Anomalie détectée	Code Erreur
#0 (Alpha-01)	sensor-01	ACCEPTÉ	Aucune, toutes les valeurs sont nominales.	Aucun
#1 (Alpha-02)	sensor-02	REJETÉ	Humidité à 150.0% (physiquement impossible).	OUT_OF_RANGE
#2 (Beta)	Vide	REJETÉ	1. sensor_id manquant. 2. Pompe sur 'on' mais	MISSING_FIELD

			débit à 0.0.	INCONSISTENT_STATE
#3 (Beta-04)	sensor-04	REJETÉ	Température à -60.0°C (hors plage tolérée).	OUT_OF_RANGE
#4 (Gamma)	sensor-05	REJETÉ	Champ timestamp manquant.	MISSING_FIELD

Rapport de performance global : Le service renvoie un statut global partial : 1 message a été correctement ingéré dans le Data Lake, tandis que 4 messages ont été isolés et documentés dans la structure JSON de la réponse.

5. Conclusion

Ce TP a permis de valider la mise en place d'une architecture logicielle robuste pour l'ingestion de flux IoT. En couplement la rigueur du typage des dataclasses au polymorphisme des validateurs, le composant est capable de filtrer efficacement la donnée de mauvaise qualité (Bad Data) à la frontière de notre système. Les mécanismes de sécurité appliqués sur les chaînes de caractères et les identifiants de sécurité garantissent la conformité totale du pipeline avec les exigences de production de l'entreprise agricole.

